Basi di dati

(aiuto!!!)

Matteo Balzerani

October 2025

Contents

I Teoria **2**

1 Introduzione **2**
- 1.1 Moduli di un DBMS 3
- 1.2 Condivisione di una BD 3
- 1.3 Gestione della memoria secondaria 4
  - 1.3.1 Buffer 4
  - 1.3.2 Gestione del buffer 4

2 Gestione delle TUPLE nelle pagine **6**
- 2.1 Strutture sequenziali 6
  - 2.1.1 Struttura seriale (disordinata) 6
  - 2.1.2 Struttura ad array 7
  - 2.1.3 Strutture ordinate (clustered) 7
- 2.2 Strutture ad accesso calcolato: HASH . . . 7
  - 2.2.1 File HASH 8
- 2.3 Strutture ad Albero 9
  - 2.3.1 Indici di file 9
- 2.4 Strutture ad albero dinamiche 11
  - 2.4.1 Albero binario di ricerca 11

3 Possibili domande d'esame **12**

Descrizione Esame

- **Modulo di Teoria** Prova scritta che consiste in domande ed esercizi sugli argomenti trattati. Possibilità di ripetere la prova. In caso di ripetizione della prova, l'ultimo voto ottenuto sostituisce il precedente.
- **Modulo di Laboratorio** Prova scritta che consiste in esercizi sugli argomenti trattati. Il voto finale dell'insegnamento di Basi di Dati e Web sarà dato dalla media tra il voto della prova scritta di teoria e il voto delle prova scritta di laboratorio. Il voto complessivo verrà pubblicato anche sulla piattaforma MyUnivr e dopo l'accettazione verrà verbalizzato.

Part I
Teoria

1 Introduzione

Cos'è un DBMS? Sistema (prodotto software) in grado di gestire collezioni di dati che siano:

- **grandi** (di dimensioni (molto) maggiori della memoria centrale dei sistemi di calcolo utilizzati). La grandezza richiede che tale gestione sia sofisticata (non possiamo caricare tutto in memoria principale e poi riscaricare).
- **persistenti** (con un periodo di vita indipendente dalle singole esecuzioni dei programmi che le utilizzano). La persistenza richiede una gestione in memoria secondaria.
- **condivise** (utilizzate da applicazioni diverse)

garantendo **affidabilità** (resistenza a malfunzionamenti hardware e software) e **privatezza** (con una disciplina e un controllo degli accessi). Come ogni prodotto informatico, un DBMS deve essere **efficiente** (utilizzando al meglio le risorse di spazio e tempo del sistema) ed **efficace** (rendendo produttive le attività dei suoi utilizzatori).

Architettura a livelli dei DBMS I DBMS sono organizzati secondo una architettura a livelli

- Livello **LOGICO**: a cui fanno riferimento gli utenti (SQL).
- Livello **FISICO**: stabilisce l'effettiva implementazione in modo efficiente

  Esempio: le relazioni del modello relazionale dei dati vengono realizzate utilizzando varie possibili strutture fisiche

La proprietà di INDIPENDENZA dei dati permette agli utenti di ignorare le strutture fisiche – SQL fa riferimento al livello logico – Il DBMS si preoccupa di tradurre le operazioni al livello più basso, utilizzando in modo efficiente le strutture fisiche. Gli utenti vedono il modello logico (relazionale).
I dati sono in **memoria secondaria**, le strutture logiche non sarebbe efficienti in memoria secondaria: servono strutture fisiche opportune. La memoria secondaria è molto più lenta della memoria principale: serve un'interazione fra memoria principale e secondaria che limiti il più possibile gli accessi alla secondaria.

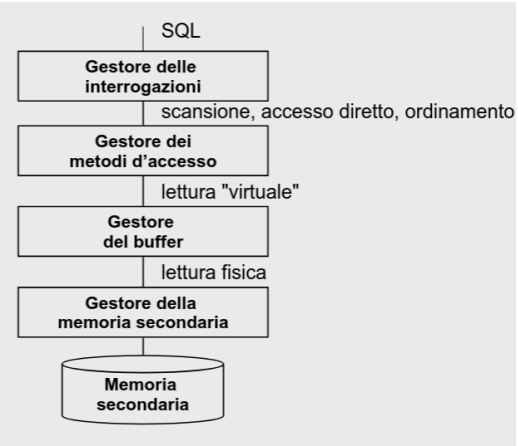


Figure 1: Architettura della gestione degli accessi e delle interrogazioni in un DBMS

1.1 Moduli di un DBMS

Domanda d'esame: ***Si descrivano i moduli che compongono il DBSM descrivendo brevemente le funzioni di ciascuno:*** risposta nelllo schema sottostante. Bisogna parlare di ciascun modulo brevemente, facendo l'esempio di una query dall'alto verso il basso e viceversa.

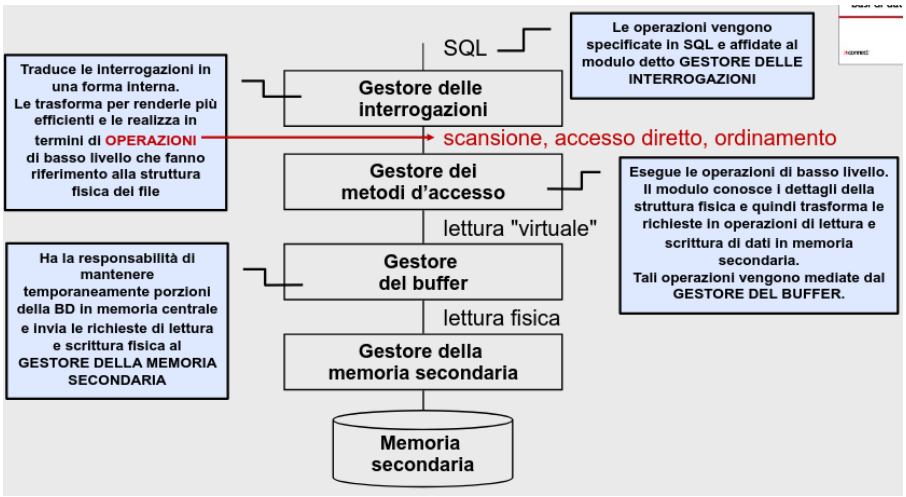


Figure 2

Gestore delle interrogazioni Prende la query logica e la traduce in una forma interna. Le trasforma per renderle più efficienti e le realizza in termini di **operazioni di basso livello** che fanno riferimento alla struttura fisica dei file.

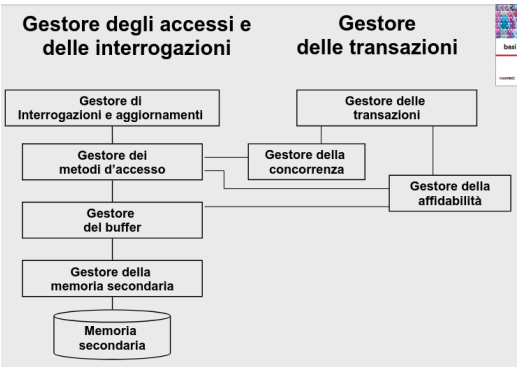
Gestore dei metodi di accesso Esegue le operazioni di basso livello. Il modulo conosce i dettagli della struttura fisica e quindi trasforma le richieste in operazioni di lettura e scrittura di dati in memoria secondaria. Tali operazioni vengono mediate dal GESTORE DEL BUFFER.

Gestore del buffer Ha la responsabilità di manetenere temporaneamente porzioni della BD in memoria centrale e invia le richieste di lettura e scrittura fisica al **gestore della memoria secondaria.**

Transazioni le basi di dati si basano sul concetto di transazione (insieme di operazioni), che deve essere **atomiche** (o tutto o niente) e **definitive** (dopo che è andata a buon fine, deve rimanere permanente la transazione eseguita).

1.2 Condivisione di una BD

Una BD è una risorsa *integrata*, *condivisa* fra le varie applicazioni, e di conseguenza vi sono attività diverse su dati in parte condivisi; questo richiede **meccanismi di autorizzazione** e attività multi-utente su dati condivisi: richiedono **controllo della concorrenza.** L'accesso concorrente in scrittura deve essere regolato.



Il **gestore della concorrenza e il gestore della affidabilità sono sottomoduli del gestore delle transazioni** (DBMS sistema informativo transazionale.) Significa gestire la concorrenza e gestire l'affidabilità.

Domanda: *Quali sono i moduli che mi permettono di gestire le transazioni in un DBMS?*

1.3 Gestione della memoria secondaria

Il costo di un accesso a memoria secondaria è quattro o più ordini di grandezza maggiore di quello per operazioni in memoria centrale. Nelle applicazioni "I/O bound" (cioè con molti accessi a memoria secondaria e relativamente poche operazioni) il costo dipende esclusivamente dal numero di accessi a memoria secondaria. Nei DBMS l'interazione tra memoria centrale e secondaria è realizzata attraverso un buffer.

1.3.1 Buffer

- Area di memoria centrale, gestita dal DBMS (preallocata) e condivisa fra le transazioni.
- Organizzato in **pagine** di dimensioni pari o multiple di quelle dei blocchi di memoria secondaria. (Per semplicità ipotizzeremo che ciascuna pagina corrisponda ad un blocco)
- L'allocazione di buffer sempre più grandi, riduce il numero di accessi alla memoria secondaria
- La gestione ottimale del buffer è un aspetto fondamentale nelle BD (rispetto alle prestazioni). Il buffer è importantissimo per via della grande differenza di tempo di accesso fra memoria centrale e memoria secondaria.

1.3.2 Gestione del buffer

Scopo della gestione del buffer

- **Ridurre il numero di accessi alla memoria secondaria** *GARANTENDO L'AFFIDABILITÀ.*
  - In caso di **lettura**, se la pagina è già presente nel buffer, non è necessario accedere alla memoria secondaria.
  - In caso di **scrittura**, il gestore del buffer può decidere di differire la scrittura fisica (ammesso che ciò sia compatibile con la gestione dell'affidabilità)
- Le politiche di gestione del buffer assomigliano a quelle della gestione della memoria centrale da parte dei Sistemi Operativi: obbediscono allo stesso principio, detto di località, in base al quale i dati referenziati di recente hanno una maggiore probabilità di essere referenziati nuovamente in futuro

Il buffer offre ai moduli di livello più alto una interfaccia complessa, non si limita alle richieste di lettura e scrittura. Ha bisogno delle informazioni sul **prevedibile riutilizzo delle pagine** (per evitarne la riapertura); Ha bisogno delle informazioni sulla necessità di **effettuare immediatamente gli aggiornamenti** (per evitare la perdita in caso di guasto). Per la gestione dei buffer sono necessarie le seguenti informazioni:

- Un direttorio che per ogni pagina mantiene (ad esempio) il file fisico e il numero del blocco ad essa corrispondente.
- Per ogni pagina del buffer il gestore mantiene due variabili di stato: **un contatore** che indica quanti programmi utilizzano la pagina e un **bit** che indica se la pagina è "sporca", cioè se è stata modificata (*dirty bit*).

Funzioni del buffer manager Le politiche sono simili a quelle relative alla gestione della memoria da parte dei sistemi operativi.

Principio di "località dei dati": è alta la probabilità di dover riutilizzare i dati attualmente in uso.
Legge 80-20: l'80% delle operazioni utilizza sempre lo stesso 20% dei dati.

Operazioni per la gestione del buffer:

- **Fix:** Operazione che permette a un processo di accedere a una pagina, caricandola dal disco solo se non è già presente nel buffer. Incrementa il contatore dei riferimenti (pin count) della pagina per bloccarla ed evitare che venga rimossa mentre è in uso. Restituisce il riferimento (indirizzo) alla pagina nel buffer.

  Passi dell'operazione:

  1. **Cerca la pagina nel buffer.**
     - Se è presente (grazie al principio di località, ciò avviene abbastanza spesso): incrementa il contatore e restituisce l'indirizzo.
     - Se non è presente: procedi al passo successivo.
  2. **Cerca un frame libero** (cioè una pagina con contatore = 0, non usata da nessun processo).
     - Se lo trova:
       * Se il dirty bit (bit di stato modificato) è 1, riscrive la pagina in memoria di massa prima di sovrascriverla.
       * Carica la nuova pagina dal disco, imposta il contatore a 1, restituisce l'indirizzo.
     - Se non trova frame liberi (tutti i contatori sono > 0): adotta una delle due strategie:
       * **Steal:** seleziona una pagina "vittima" (es. con algoritmo LRU), la scrive su disco se modificata, la sovrascrive con la nuova pagina, restituisce l'indirizzo.
       * **No-steal:** mette l'operazione in attesa finché non si libera un frame (cioè qualche processo esegue Unfix portando un contatore a 0).

- **setDirty** Comunica al buffer manager che la pagina è stata modificata. Viene modificato il BIT di stato relativo.

- **Unfix:** indica che la transazione ha concluso l'utilizzo della pagina, decrementa il contatore associato alla pagina.

- **Force:** Trasferisce in modo sincrono una pagina in memoria secondaria **su richiesta del gestore dell'affidabilità** (non del gestore degli accessi) per garantire che alcuni dati non vengano persi. Memorizza in maniera permanente le modifiche per garantire l'affidabilità.

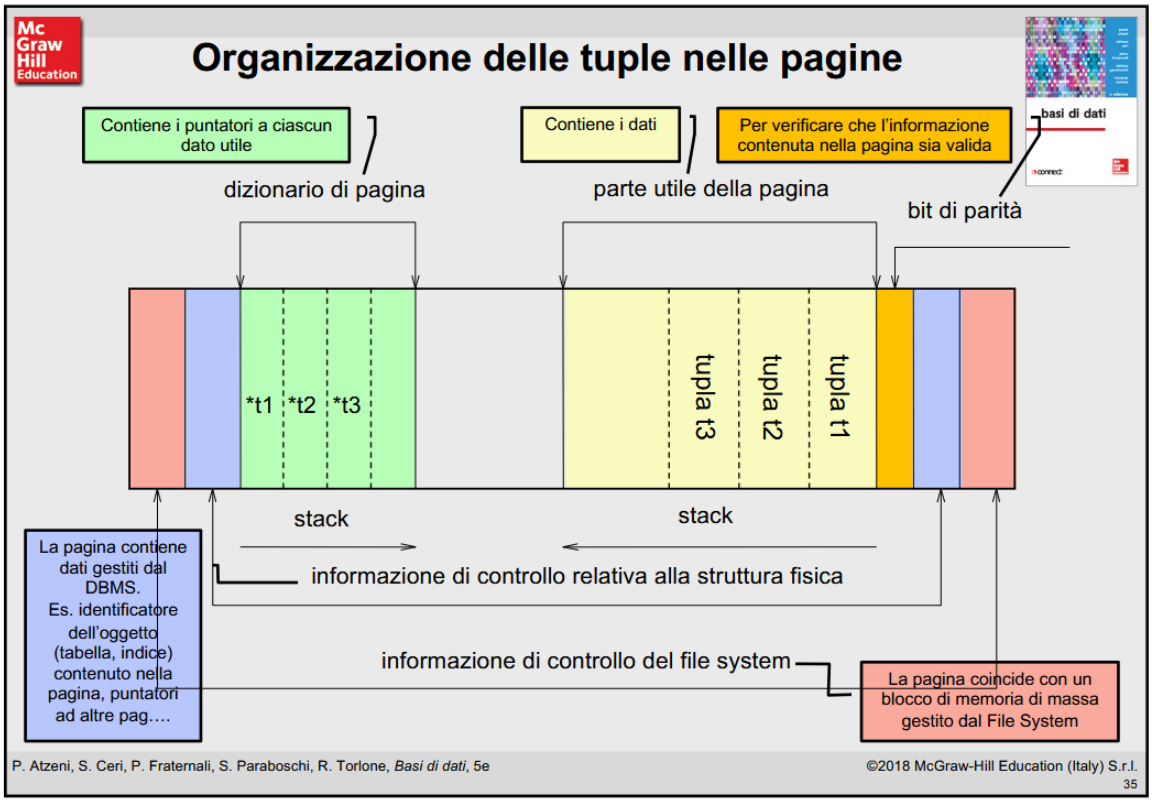
La scrittura in memoria secondaria può avvenire in due modi:

- in modo **sincrono** quando è richiesto esplicitamente con una force
- in modo **asincrono** (indipendente dall'applicazione) quando il buffer manager lo ritiene opportuno (o necessario). In particolare, può decidere di anticipare o posticipare scritture per coordinarle e/o sfruttare la disponibilità dei dispositivi

2 Gestione delle TUPLE nelle pagine

Ciascuna pagina contiene l'*informazione utile*, cioè i dati veri e proprio, e l'*informazione di controllo*, per accedere all'informazione utile. I blocchi (componenti fisici di un file) e i record (componenti logici) hanno dimensioni in generale diverse: la dimensione del blocco *dipende dal file system* e la dimensione del record dipende dalle esigenze del record.

Fattore di blocco Il numero di **record** in un **blocco** è pari al rapporto fra la dimensione del blocco L_B e la dimensione media dei record L_R. Se $L_B > L_R$ possiamo avere più record in un blocco:$\lfloor L_B/L_R \rfloor$. Lo spazio residuo può essere *utilizzato*, cioè **spanned** (impaccato) o non utilizzato, cioè **unspanned**



Abbiamo diverse strutture primarie per l'organizzazione di file, e stabiliscono il *criterio secondo cui le tuple sono disposte nel file*. Ci sono tre categorie principali di strutture:

- **Sequenziali:** disposizione consecuitva delle tuple
- **Ad accesso calcolato - HASH:** disposizione in posizioni che dipendono dall'esecuzione di un algoritmo
- **Ad albero:** usate soprattutto come **strutture secondarie.**

2.1 Strutture sequenziali

Nelle strutture sequenziali un file è costituito da vari blocchi di memoria "logicamente" consecutivi. Le tuple vengono inserite nei blocchi rispettando una sequenza.

2.1.1 Struttura seriale (disordinata)

È la struttura sequenziale più semplice ma anche più diffusa, le tuple venogno inserite nel file nell'ordine in cui si presentano; non esiste un ordinamento basato su qualche valore. Viene anche chiamata **"entry sequenced"**/file heap/file disordinato. Gli inserimento vengono effettuati *in coda* o *al posto di record cancellati*. Le **cancellazioni** si effettuano marcando il record come "cancellato". le tuple vengono periodicamente riorganizzate per eliminare lo spazio sprecato.
È molto diffusa nelle basi di dati relazionali, associata ad inidici secondari. È molto efficiente per le operazioni di inserimento, poiché è sufficiente mantenere un riferimento all'ultimo blocco. Le *ricerche sono poco efficienti:* richiedono una scansione sequenziale di tutti i record, con un costo lineare nel numero di blocchi del file. Per questo motivo sono spesso usate insieme a strutture secondarie usate per favorire gli accessi.

2.1.2 Struttura ad array

È possibile solo quando le tuple hanno lunghezza fissa: al file viene associato un numero di blocchi contigui; ogni *blocco* ha un numero fisso di posizioni disponibili per le tuple; ogni *tupla* ha un indice i e viene posta nella posizione dell'indice associato. Questa struttura non è quasi mai utilizzata dai DBMS anche se sono efficienti per la ricerca.

2.1.3 Strutture ordinate (clustered)

Prevedono la memorizzazione dei record secondo un **ordinamento fisico coerente con l'ordinamento di un campo detto CHIAVE.** Permettono ricerche binarie, ma solo fino ad un certo punto, poiché senza indici non conosci la disposizione dei blocchi, quindi non puoi *calcolare la posizione a metà di un file.* Per questo, nelle basi di dati relazionali si utilizzano quasi solo in combinazioni con indici come **file ISAM** o file ordinati con *indice primario.* ISAM è l'acronimo di *Indexed Sequential Access Method.*

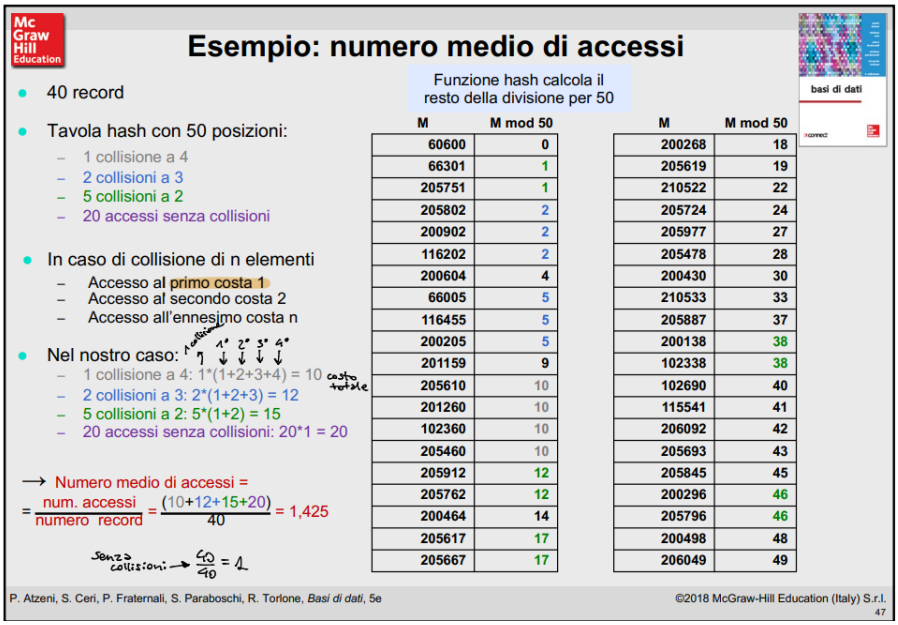
2.2 Strutture ad accesso calcolato: HASH

Le tuple sono collocate in posizioni che dipendono dall'esecuziuone di un algoritmo. Permettono un accesso diretto molto efficiente. La tecnica si basa su quella utilizzata per le tavole hash in memoria centrale.
Obiettivo: accesso diretto ad un insieme di record sulla base del valore di un campo (detto chiave, che per semplicità supponiamo identificante, ma non è necessario).

- Se i possibili valori della chiave sono in numero paragonabile al numero di record (e corrispondono ad un "tipo indice") allora usiamo un array. *Esempio: università con 1000 studenti e numeri di matricola compresi fra 1 e 1000 o poco più e file con tutti gli studenti.*
- Se i possibili valori della chiave sono molti di più di quelli effettivamente utilizzati, non possiamo usare l'array (spreco); *ad esempio: 40 studenti e numero di matricola di 6 cifre (un milione di possibili chiavi).*

Volendo continuare ad usare qualcosa di simile ad un array,ma senza sprecare spazio, possiamo pensare di trasformare i valori della chiave in possibili indici di un array: **Funzione HASH**.

Funzione HASH Associa ad ogni valore della chiave un "indirizzo", in uno spazio di dimensione paragonabile (leggermente superiore) rispetto a quello strettamente necessario. La funzione non può essere iniettiva, quindi il numero di possibili chiavi è molto maggiore del numero di possibili indirizzi ("lo spazio delle chiavi è più grande dello spazio degli indirizzi"), ed esiste la possibilità di collisioni (chiavi diverse che corrispondono allo stesso indirizzo). Le buone funzioni hash distribuiscono in modo causale e uniforme, riducendo le probabilità di collisione (che si riduce aumentando lo spazio ridondante). Varie tecniche: posizioni successive disponibili, tabella di overflow (gestita in forma collegata), funzioni hash "alternative".



Nota:

- le collisioni ci sono (quasi) sempre
- le collisioni multiple hanno probabilità che decresce al crescere della molteplicità
- la molteplicità media delle collisioni è molto bassa

2.2.1 File HASH

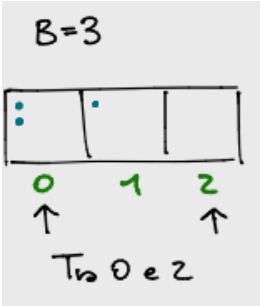
L'idea è la stessa della tavola hash, ma si basa sull'organizzazione in blocchi. L'operazione di costo unitario è l'accesso ad un blocco che può contenere diversi record. I blocchi sono riempiti solo in parte:

- **Fattore di riempimento FdR:** frazione dello spazio fisico disponibile mediamente utilizzato. *Ad esempio: 40 record di studenti su 50 posizioni disponibili* $\rightarrow FdR = 40/50 = 0.80$
- **Fattore di blocco FdB:** rapporto fra la dimensione del blocco L_B e la dimensione del record L_R: $FdB = \lfloor L_B/L_R \rfloor$

Se supponiamo di avere:

- **T:** numero di tuple previsto per il file
- **FdB:** fattore di blocco
- **FdR:** fattore di riempimento

Allora il file può avere un numero di **Blocchi B** pari a $B = \lceil \frac{T}{FdR \cdot FdB} \rceil$
La funzione di HASH ritorna un numero compreso tra 0 e $B - 1$, interpretato come il numero del blocco nell'ambito dei blocchi allocati al file.



I **blocchi allocati al file** sono **B**, la funzione di HASH ritorna un numero tra **0 e B-1**. In caso di collisione i record vengono semplicemente allocati nel blocco fino ad esaurimento dello spazio libero; vengono accettate FdB (numero di record nel blocco) collisioni. Quando un blocco viene esaurito viene allocato un nuovo blocco collegato al precedente ed il record viene disposto nel nuovo blocco: *CATENE DI OVERFLOW*.

È **l'organizzazione più efficiente per l'accesso diretto basato su valori della chiave** con condizioni di uguaglianza (accesso puntuale): il costo medio è di poco superiore all'unità e il caso peggiore è molto costoso ma talmente improbabile da poter essere ignorato. *Le collisioni (overflow) sono di solito gestite con blocchi collegati.* **Non è efficiente per ricerche basate su intervalli**: le funzioni HASH tendono a collocare valori vicini su blocchi diversi, ad esempio: ricerca degli studenti con matricola compresa tra 17010 e 17029 richiede 20 accessi diretti uno per ciascun valore dell'intervallo.

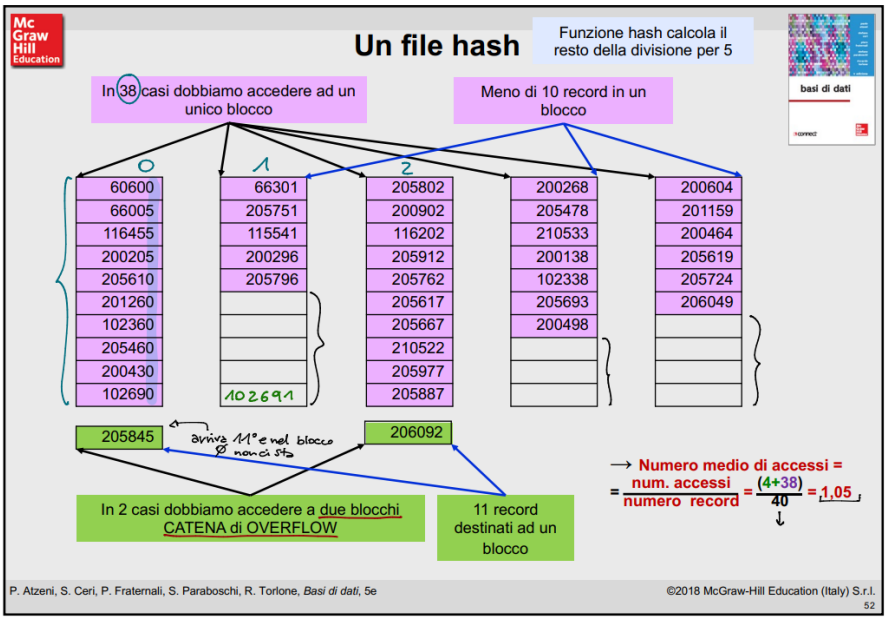


Figure 3: Esempio File hash. Numero tuple T = 40. Numero Blocchi B = 5.FdB = 10 (10 posizioni in ogni blocco). Posizioni disponibili = 50. FdR = 0.80. La funzione di HASH restituisce un numero tra 0 e 4. Numero di Overflow 2 → num. medio di accessi: 1,05

2.3 Strutture ad Albero

L'organizzazione ad albero può essere utilizzata per realizzare *strutture primarie (contenenti i dati)* e *strutture secondarie*, che favoriscono l'accesso ai dati senza contenere i dati stessi.

> È leggermente *meno efficiente* per accessi puntuali, ma mantiene la stessa efficienza anche per accessi basati su interalli.

2.3.1 Indici di file

Indice: struttura ausiliaria per l'*accesso efficiente* ai record di un file sulla base dei valori di un campo (o di una concatenazione di campi) detto **chiave** (o, meglio, pseudochiave, perché non è necessariamente identificante).

> **Idea fondamentale:** l'indice analitico di un libro, lista di coppie (termine, pagina), ordinata alfabeticamente sui termini, posta in fondo al libro e separabile da esso.

Un indice I di un file f è un **altro file**, con **record a due campi:** *chiave* e *indirizzo* (dei record di f o dei relativi blocchi), *ordinato* secondo i valori della chiave.

Tipi di indice A seconda della popolazione dell'indice abbiamo:

- **Indice denso:** contiene un record per ciascun valore del campo chiave.
- **Indice sparso:** contiene un numero di record *inferiore* rispetto al numero di valori diversi dal campo chiave, cioè *esistono valori della chiave che NON sono presenti nell'indice.*

Vi è poi la differenza tra:

- **Indice primario:** l'indice contiene al suo interno i dati o è realizzato su un file ordinato sullo stesso capmo su cui è definito l'indice stesso. *Può essere sparso.*

  Esempio: indice generale di un libro, determina la posizione dei capitoli che sono i dati del libro.

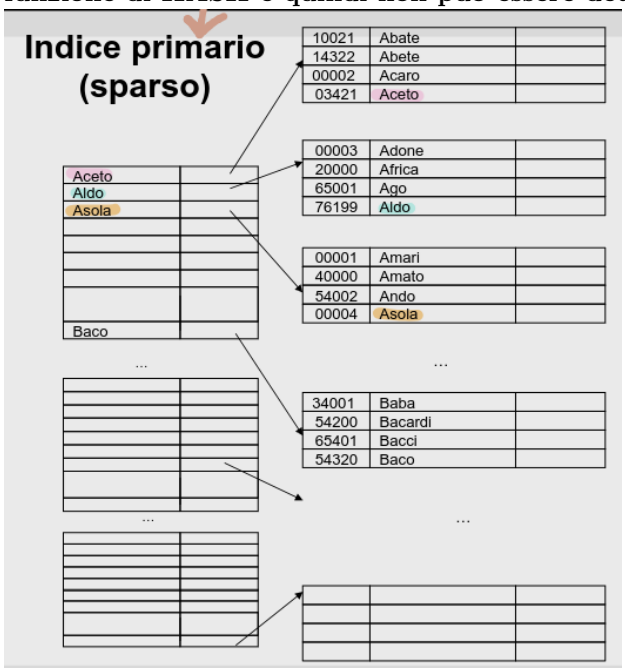
  Un indice primario, grazie all'ordinamento, può essere realizzato "puntando" ad un solo record di un blocco (indice sparso), poiché gli altri record sono posti in modo adiacente a quello puntato.

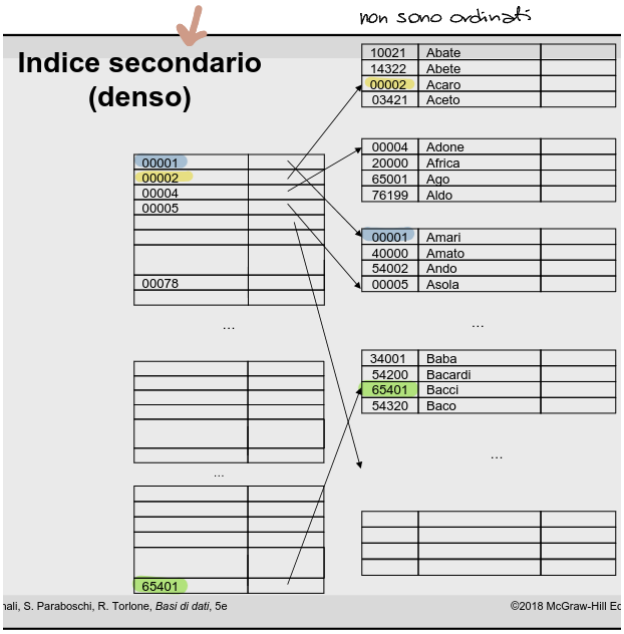
- **Indice secondario:** l'indice è definito su un campo con ordinamento diverso da quello di memorizzazione. **DEVE essere denso.**

  Esempio: indice analitico di un libro, che è l'indice secondario che garantisce l'accesso senza contenere i dati.

  Un indice secondario deve contenere tutti i rifermenti a tutti i valori della chiave (indice denso), infatti valori consecutivi della chiave possono trovarsi in blocchi diversi.

I benefici legati alla presenza di indici secondari sono molto più sensibili. *Ogni file può avere al più un indice primario e un numero qualunque di indici secondari (su campi diversi)..*
Un file hash non può avere un indice primario: Il criterio di memorizzazione è stabilito dalla funzione di HASH e quindi non può essere determinato dall'indice.





Domanda d'esame: si dia la definizione di indice, che cos'è un indice primario, secondario, denso e sparso.
*Risposta:*l'indice è un file che ha un insieme di coppie con valore di chiave e indirizzo, se l'indice contiene le informazioni è un indice primario, se non contiene le informazioni è secondario. Se contiene un Record per ogni valore della chiave è denso, se no è sparso. Gli indici primari possono essere sparsi. Gli indici secondari devono essere densi.

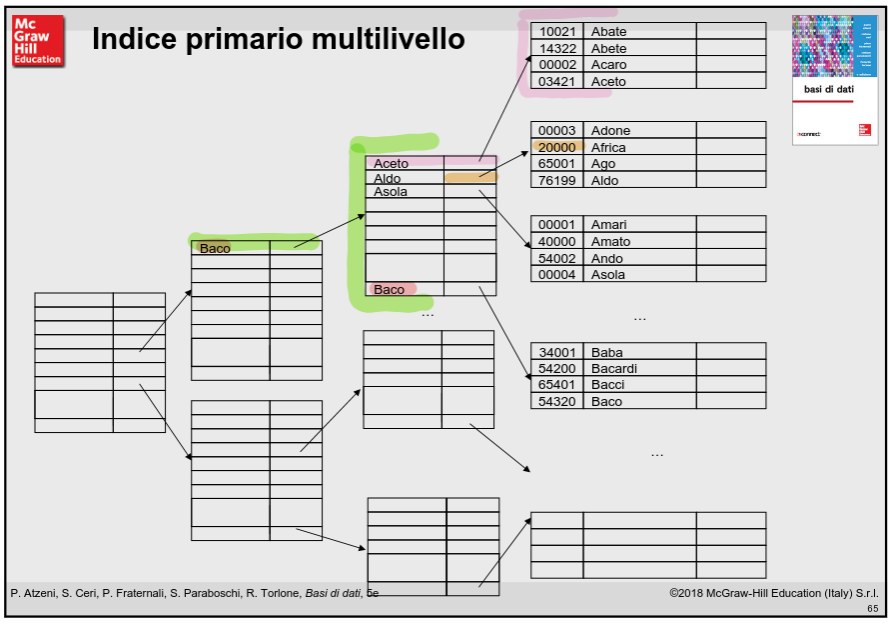
Prestazioni degli indici

- Accesso diretto sulla chiave efficiente, sia puntuale sia per intervalli
- scansione sequenziale ordinata efficiente: tutti gli indici (in particolare quelli secondari) forniscono un **ordinamento logico** sui record dei file.
- Modifiche della chiave, inserimenti, eliminazioni *inefficienti*: esistono tecniche per alleviare i problemi: *file o blocchi di overflow, marcatura per le eliminiazioni, riempimento parziale, blocchi collegati (non contigui), riorganizzazioni periodiche...*

Osservazione sugli indici secondari: si possono usare, come detto, puntatori ai blocchi oppure puntatori ai record:

- I puntatori ai blocchi sono più compatti
- I puntatori ai record permettono di semplificare alcune operazioni (effettuate solo sull'indice, senza accedere al file se non quando indispensabile)

Indici multilivello Gli indici sono file essi stessi e quindi ha senso costruire indici sugli indici, per evitare di fare ricerche fra blocchi diversi. Possono esistere più livelli fino ad avere il livello più alto con un solo blocco. I livelli sono di solito abbastanza pochi, perché l'indice è ordinato, quindi l'indice sull'indice è sparso, e i record dell'indice sono piccoli.



Tutte le strutture di indice viste finora sono basate su strutture ordinate e quindi sono poco flessibili in presenza di elevata dinamicità. Gli indici utilizzati dai DBMS sono più sofisticati. Si usano strutture al albero dinamiche. Sono efficienti in presenza di aggiornamenti.

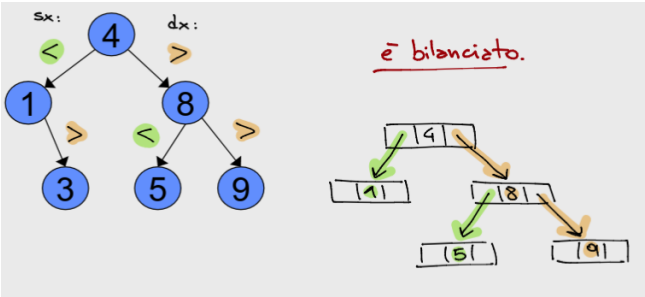
2.4 Strutture ad albero dinamiche

Ogni albero è caratterizzato da un **nodo radice**, vari **nodi intermedi**, e vari **nodi foglia**. Ogni nodo coincide con una *pagina o blocco* a livello di file system e di gestore del buffer. Ogni nodo ha un numero di figli abbastanza grande che dipende dall'ampiezza della pagina. I legami tra i nodi vengono stabiliti da *puntatori* che collegano fra loro le pagine.

> Requisito importante per il buon funzionamento di queste strutture è che devono essere **bilanciate**, ovvero che la *lunghezza del cammino che collega la radice alle foglie è* ***costante***, ovvero che il tempo di accesso è uguale per tutte le informazioni.

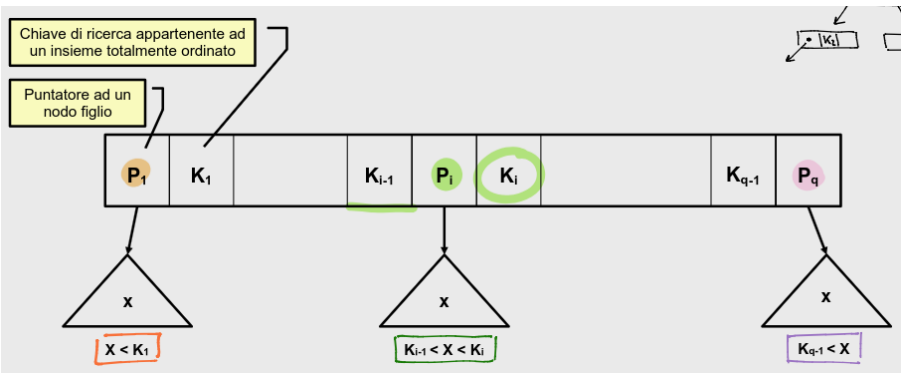
2.4.1 Albero binario di ricerca

È un albero binario etichettato in cui per ogni nodo il *sottoalbero sinistro* contiene solo **etichette** *minori di quella del nodo* e il *sotttoalbero destro* etichette maggiori. **Tempo di ricerca ed inserimento pari alla profondità.** Logaritmico nel caso "medio".



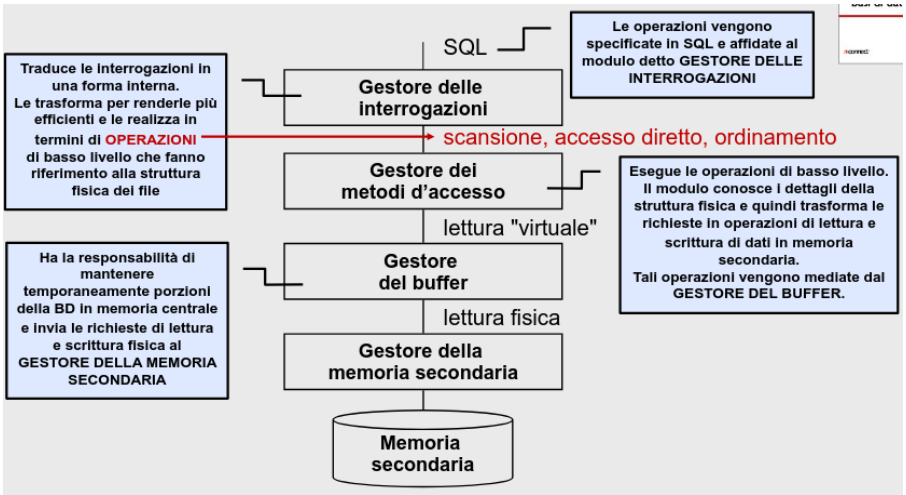
2.4.2 Albero di ricerca di ordine Q

Ogni nodo ha fino a **Q figli** e fino a **Q-1** etichette, ordinate.
Nell'i-esimo sottoalbero abbiamo tutte etichette magiori della (i-1)-esima etichetta e minori della i-esima.


Chiave di ricerca appartenente ad un insieme totalmente ordinato
Puntatore ad un nodo figlio
P1
K1
Ki-1
Pi
Ki
Kq-1
Pq
x
x
x
X < K1
Ki-1 < X < Ki
Kq-1 < X

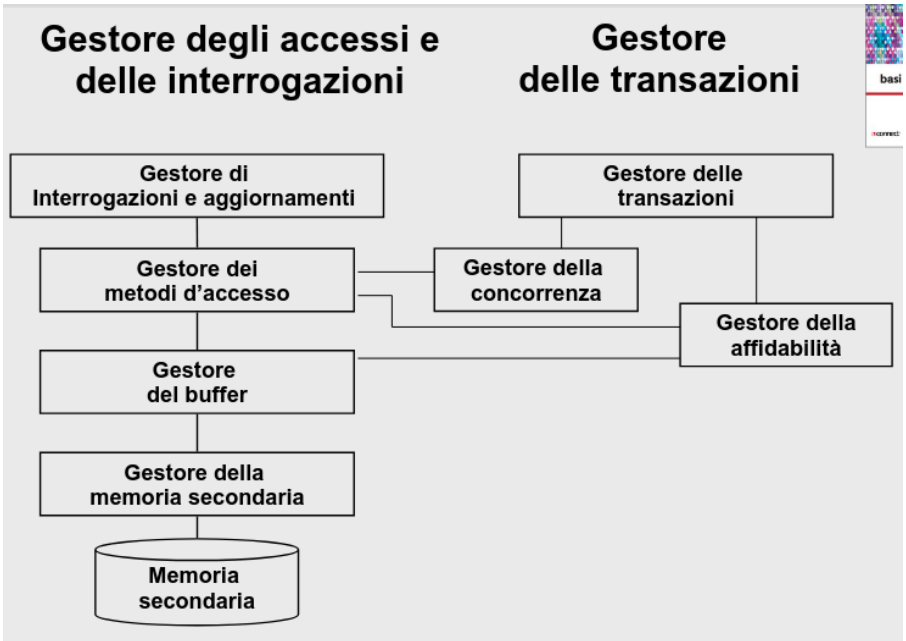
3 Possibili domande d'esame

- ***Si descrivano i moduli che compongono il DBSM descrivendo brevemente le funzioni di ciascuno:*** risposta nelllo schema sottostante.



Bisogna parlare di ciascun modulo brevemente, facendo l'esempio di una query dall'alto verso il basso e viceversa.

- ***(da approfondire) Quali sono i moduli che mi permettono di gestire le transazioni in un DBMS?*** Significa gestire la concorrenza e gestire l'affidabilità.



- ***Si descriva in che modo le tuple vengono organizzate nelle pagine:***

B + Tree F Chiavi, ogni chiave è seguita da un puntatore. La chiave K_1 ha un puntatore prima di sè P_0, quindi abbiamo $F+1$ puntatori che si chiama **Fan Out dell'albero**. (Dato Fan Out =4, abbiamo 3 chiavi). I puntatori saranno sempre uno in più, il Fan Out dice il numero massimo di puntatori che posso avere. In questo caso il puntatore P_0 che sta prima della prima chiave punta al sottoalbero che contiene le chiavi $K < K_1$. P_F punta al sottoalbero che contiene le chiavi $K \geq K_F$ mentre P_i è il puntatore che punta al sottoalbero che contiene le chiavi $K_i \leq K < K_{i+1}$. Data la chiave di ricerca V, ad ogni nodo intermedio, se $V < K$ allora si segue il puntatore P_0, se $V \geq K_F$ si segue il puntatore P_F, altrimenti si segue il puntatore P_j tale che $K_j \leq K < K_{j+1}$. Se l'indice è **primario** i nodi foglia contengono l'intera tupla, se l'indice è secondario i nodi foglia contengono i putnatori ai blocchi della BD che contengono le tuple.
Inserimenti e cancellazioni: nel caso di inserimento o eliminazione di una tupla devo risistemare l'albero affinché rispetti i vincoli. Se c'è posto nella foglia lo inserisco. Se arrivo in un nodo con numero di chiavi

massimo devo dividere il nodo, splittarlo, in modo da inserirlo, ovviamente aggiungendo un nuovo puntatore. Eliminazione, c'è la possibilità di fare un merge. Se devo modificare il valore di una chiave, devo trattarle come eliminazioni seguite da inserimenti.